آشنایی با کلاس‌ها و برنامه‌نویسی شیءگرا در پایتون

تا اینجا برنامه‌های خود را با متغیرها، ساختارهای داده و توابع نوشته‌ایم. اما وقتی یک برنامه بزرگ‌تر می‌شود، ممکن است بخواهیم اطلاعات و عملیات مرتبط با یک موضوع را کنار هم نگهداری کنیم.

برای مثال، در یک برنامه فروشگاهی، هر محصول نام، قیمت و موجودی دارد و می‌توان عملیاتی مانند محاسبه تخفیف یا تغییر موجودی را روی آن انجام داد.

پایتون برای سازمان‌دهی چنین برنامه‌هایی از کلاس‌ها (Classes) و اشیا (Objects) پشتیبانی می‌کند. این شیوه برنامه‌نویسی را برنامه‌نویسی شیءگرا یا Programming Object-Oriented (OOP) می‌نامیم.

۱. کلاس و شیء چیست؟

برای درک مفهوم کلاس، یک قالب یا نقشه را در نظر بگیرید.

فرض کنید می‌خواهیم اطلاعات چند محصول را در برنامه نگهداری کنیم. همه محصولات ویژگی‌های مشابهی دارند، مانند نام و قیمت؛ اما مقدار این ویژگی‌ها برای هر محصول متفاوت است.

در برنامه‌نویسی شیءگرا:

- کلاس (Class) قالبی است که مشخص می‌کند یک نوع شیء چه ویژگی‌ها و رفتارهایی دارد.
- شیء (Object) نمونه‌ای است که بر اساس یک کلاس ساخته می‌شود.
- ویژگی (Attribute) اطلاعاتی است که یک شیء نگهداری می‌کند.
- متد (Method) تابعی است که در یک کلاس تعریف شده و رفتاری را برای آن مشخص می‌کند.

برای مثال، `Product` می‌تواند یک کلاس باشد و هر محصول واقعی که با استفاده از آن می‌سازیم، یک شیء از این کلاس خواهد بود.

۲. تعریف یک کلاس ساده

در پایتون، برای تعریف کلاس از کلمه کلیدی `class` استفاده می‌کنیم.

```
class Product:
    pass
```

در این مثال، کلاسی به نام `Product` تعریف کرده‌ایم. کلمه `pass` فعلاً به این معناست که بدنه کلاس خالی است.

نام کلاس‌ها معمولاً با حرف بزرگ شروع می‌شود و اگر از چند کلمه تشکیل شده باشند، هر کلمه را با حرف بزرگ آغاز می‌کنیم؛ مانند `ShoppingCart`.

اکنون می‌توانیم از این کلاس یک شیء بسازیم:

```
class Product:
    pass


product1 = Product()
product2 = Product()
```

در این برنامه، دو شیء مستقل از کلاس `Product` ساخته‌ایم و آن‌ها را در متغیرهای `product1` و `product2` قرار داده‌ایم.

البته این کلاس هنوز ویژگی یا رفتار مشخصی ندارد. در ادامه آن را کامل می‌کنیم.

۳. ویژگی‌های یک شیء

می‌توانیم اطلاعات مرتبط با هر شیء را در ویژگی‌های آن نگهداری کنیم.

برای مثال، می‌خواهیم هر محصول نام و قیمت داشته باشد:

```python
class Product:
    pass


product1 = Product()
product1.name = "Laptop"
product1.price = 1200

product2 = Product()
product2.name = "Mouse"
product2.price = 25

print(product1.name)
print(product1.price)

print(product2.name)
print(product2.price)
```

خروجی:

```
Laptop
1200
Mouse
25
```

در این مثال، هر دو شیء از یک کلاس ساخته شده‌اند، اما اطلاعات متفاوتی دارند.

در پایتون می‌توان بسیاری از ویژگی‌ها را پس از ساختن شیء به آن اضافه کرد. بااین‌حال، برای ساختار منظم‌تر، معمولاً ویژگی‌های اصلی را هنگام ایجاد شیء تعریف می‌کنیم.

۴. سازنده کلاس و متد `__init__`

برای تعیین ویژگی‌های اولیه یک شیء، از متد ویژه `__init__()` استفاده می‌کنیم.

این متد هنگام ساختن شیء، برای مقداردهی اولیه آن فراخوانی می‌شود.

مثال زیر نسخه کامل‌تری از کلاس `Product` است:

```python
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price


product1 = Product("Laptop", 1200)
product2 = Product("Mouse", 25)

print(product1.name)
print(product2.name)
```

خروجی:

```
Laptop
Mouse
```

در این مثال:

- `__init__()` اطلاعات اولیه موردنیاز محصول را دریافت می‌کند.
- `name` و `price` پارامترهایی هستند که هنگام ساخت شیء به تابع داده می‌شوند.

- `self.name` و `self.price` ویژگی‌های شیء را مشخص می‌کنند.
- هنگام ساختن هر محصول، مقادیر مناسب به آن اختصاص داده می‌شوند.

به کمک `__init__()`، دیگر لازم نیست پس از ساختن هر شیء، ویژگی‌های آن را جداگانه تنظیم کنیم.

نکته: در گفت‌وگوهای آموزشی معمولاً به `__init__()` سازنده می‌گویند؛ دقیق‌تر آن است که بدانیم این متد وظیفه مقداردهی اولیه شیء ساخته‌شده را بر عهده دارد.

۵. `self` چیست؟

یکی از مفاهیم مهم هنگام یادگیری کلاس‌ها در پایتون، `self` است.

این پارامتر به نمونه‌ای از کلاس اشاره می‌کند که متد در حال کار با آن است.

برای مثال:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
```

وقتی می‌نویسیم:

```
product1 = Product("Laptop", 1200)
product2 = Product("Mouse", 25)
```

در جریان مقداردهی اولیه `product1`، عبارت `self.name` به ویژگی نام همان شیء اشاره می‌کند. هنگام ساخت `product2` نیز همین کد برای شیء دوم اجرا می‌شود.

به این ترتیب، هر شیء اطلاعات مستقل خود را دارد.

در پایتون، `self` یک کلمه کلیدی ویژه نیست، اما نام قراردادی و استانداردی برای این پارامتر است. هنگام تعریف متدهای نمونه، معمولاً باید آن را به‌عنوان نخستین پارامتر بنویسیم.

هنگام فراخوانی متد، لازم نیست خودمان مقدار `self` را ارسال کنیم؛ پایتون نمونه مربوط را به‌صورت خودکار در اختیار متد قرار می‌دهد.

۶. متدها؛ رفتارهای یک شیء

تا اینجا شیءها اطلاعاتی مانند نام و قیمت محصول را نگهداری می‌کردند. اکنون می‌توانیم رفتارهایی را نیز برای آن‌ها تعریف کنیم.

برای مثال، می‌خواهیم هر محصول بتواند اطلاعات خود را نمایش دهد و قیمت پس از تخفیف را محاسبه کند.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def display_info(self):
        print(f"Product: {self.name}")
        print(f"Price: {self.price}")

    def calculate_discounted_price(self, discount_percent):
        discount = self.price * discount_percent / 100
        return self.price - discount


product = Product("Laptop", 1200)
```

```
product.display_info()

discounted_price = product.calculate_discounted_price(10)
print(f"Discounted price: {discounted_price}")
```

خروجی:

```
Product: Laptop
Price: 1200
Discounted price: 1080.0
```

در این مثال، دو متد تعریف کرده‌ایم:

- `display_info()` اطلاعات محصول را نمایش می‌دهد.
- `calculate_discounted_price()` قیمت پس از تخفیف را محاسبه و برمی‌گرداند.

هر دو متد به اطلاعات محصول، مانند `self.name` و `self.price`، دسترسی دارند.

در نتیجه، اطلاعات و رفتارهای مرتبط با محصول در یک کلاس کنار هم قرار گرفته‌اند.

۷. چند شیء از یک کلاس

یکی از مزیت‌های کلاس‌ها این است که می‌توانیم تعداد زیادی شیء با ساختار مشابه، اما اطلاعات متفاوت، بسازیم.

برای مثال:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def calculate_discounted_price(self, discount_percent):
        discount = self.price * discount_percent / 100
        return self.price - discount


products = [
    Product("Laptop", 1200),
    Product("Mouse", 25),
    Product("Keyboard", 75)
]

for product in products:
    print(product.name, product.calculate_discounted_price(10))
```

خروجی:

```
Laptop 1080.0
Mouse 22.5
Keyboard 67.5
```

در این مثال، چند شیء از کلاس `Product` ساخته‌ایم و آن‌ها را در یک لیست قرار داده‌ایم.

سپس با استفاده از حلقه `for` روی محصولات حرکت کرده‌ایم و قیمت تخفیف‌خورده هر کدام را نمایش داده‌ایم.

این روش در برنامه‌هایی مانند فروشگاه اینترنتی، سیستم مدیریت انبار و نرم‌افزارهای حسابداری کاربرد دارد.

۸. برنامه‌نویسی شیءگرا چه کمکی به ما می‌کند؟

برنامه‌نویسی شیءگرا راهی برای سازمان‌دهی کدها و مدیریت پیچیدگی برنامه است. البته استفاده از کلاس‌ها برای تمام برنامه‌ها ضروری نیست.

چند مفهوم مهم در برنامه‌نویسی شیءگرا عبارت‌اند از:

کپسوله‌سازی (Encapsulation)

کپسوله‌سازی به سازمان‌دهی داده‌ها و رفتارهای مرتبط در یک واحد اشاره دارد و به ما کمک می‌کند نحوه استفاده از آن‌ها را مشخص کنیم.

برای مثال، در کلاس `Product`، ویژگی‌های محصول و متدی که قیمت تخفیف‌خورده را محاسبه می‌کند، در یک کلاس قرار گرفته‌اند.

در پایتون، کپسوله‌سازی بیشتر بر قراردادهای برنامه‌نویسی و روش دسترسی به داده‌ها تکیه دارد؛ برخلاف برخی زبان‌ها که کنترل دسترسی سخت‌گیرانه‌تری دارند.

وراثت (Inheritance)

وراثت به ما اجازه می‌دهد کلاسی بر اساس کلاس دیگری تعریف کنیم و ویژگی‌ها یا رفتارهای آن را به ارث ببریم.

برای مثال، در یک فروشگاه می‌توان کلاس عمومی `Product` را داشت و کلاس‌های تخصصی‌تری برای محصولات دیجیتال یا فیزیکی تعریف کرد.

وراثت زمانی مفید است که رابطه منطقی «نوعی از» میان کلاس‌ها وجود داشته باشد. بااین‌حال، برای شروع کار لازم نیست همیشه از آن استفاده کنیم.

چندریختی (Polymorphism)

چندریختی یعنی بتوانیم از یک رابط یا روش فراخوانی یکسان برای اشیایی با رفتارهای متفاوت استفاده کنیم.

برای مثال، انواع مختلف محصولات می‌توانند متدی به نام `calculate_shipping()` داشته باشند؛ اما هر نوع محصول هزینه ارسال را به شیوه متفاوتی محاسبه کند.

در این حالت، کد استفاده‌کننده می‌تواند متد را با نامی یکسان فراخوانی کند، بدون اینکه لازم باشد تمام جزئیات محاسبه را بداند.

این مفاهیم به ما کمک می‌کنند برنامه‌های پیچیده‌تر را ساختارمند طراحی کنیم؛ اما تسلط بر آن‌ها به تمرین بیشتری نیاز دارد.

۹. چه زمانی از کلاس استفاده کنیم؟

کلاس‌ها برای همه مسئله‌ها ضروری نیستند.

اگر برنامه فقط قرار است چند عدد را جمع کند یا تبدیل ساده‌ای روی داده‌ها انجام دهد، استفاده از یک تابع ممکن است کافی و حتی خواناتر باشد.

اما زمانی که با موجودیت‌هایی سروکار داریم که اطلاعات و رفتارهای مرتبط دارند، کلاس می‌تواند انتخاب مناسبی باشد.

برای مثال:

- در برنامه فروشگاهی، هر محصول می‌تواند نام، قیمت و موجودی داشته باشد.
- در برنامه مدیریت کاربران، هر کاربر می‌تواند نام و وضعیت حساب داشته باشد.
- در یک بازی، هر شخصیت می‌تواند موقعیت، انرژی و رفتارهای مشخصی داشته باشد.

بهتر است ابتدا مسئله را بفهمیم و سپس تصمیم بگیریم که آیا استفاده از کلاس واقعاً به سازمان‌دهی بهتر برنامه کمک می‌کند یا خیر.

جمع‌بندی

کلاس‌ها قالبی برای تعریف ویژگی‌ها و رفتارهای اشیا هستند. با ساختن شیء از یک کلاس، می‌توانیم داده‌های مستقل داشته باشیم و با استفاده از متدها عملیاتی را روی آن‌ها انجام دهیم.

مفاهیمی مانند `class`، `__init__()`، `self`، ویژگی‌ها و متدها پایه درک برنامه‌نویسی شیءگرا در پایتون هستند.

در پروژه‌های بزرگ‌تر، مفاهیمی مانند کپسوله‌سازی، وراثت و چندریختی به سازمان‌دهی بهتر کد کمک می‌کنند. بااین‌حال، انتخاب میان تابع‌ها و کلاس‌ها باید بر اساس نیاز واقعی برنامه انجام شود.

تمرین

کلاسی به نام `Book` بنویسید که اطلاعات یک کتاب را نگهداری کند.

این کلاس باید:

1. در متد `__init__()` عنوان کتاب، نام نویسنده و قیمت را دریافت کند.
2. اطلاعات را در ویژگی‌های `title`، `author` و `price` ذخیره کند.
3. متدی به نام `display_info()` داشته باشد که اطلاعات کتاب را نمایش دهد.
4. متدی به نام `apply_discount()` داشته باشد که درصد تخفیف را دریافت کند و قیمت پس از تخفیف را برگرداند.

سپس دو شیء از این کلاس بسازید و اطلاعات و قیمت تخفیف‌خورده آن‌ها را نمایش دهید.

تمرین تکمیلی: از یک ابزار هوش مصنوعی بخواهید کد خود را بررسی کند و پیشنهاد دهد آیا ساختار کلاس مناسب است یا می‌توان مسئله را ساده‌تر حل کرد. پیشنهادها را بررسی کنید و دلیل انتخاب خود را توضیح دهید.